

Hands-on Lab: Static Code Analysis



Estimated time needed: **30** minutes

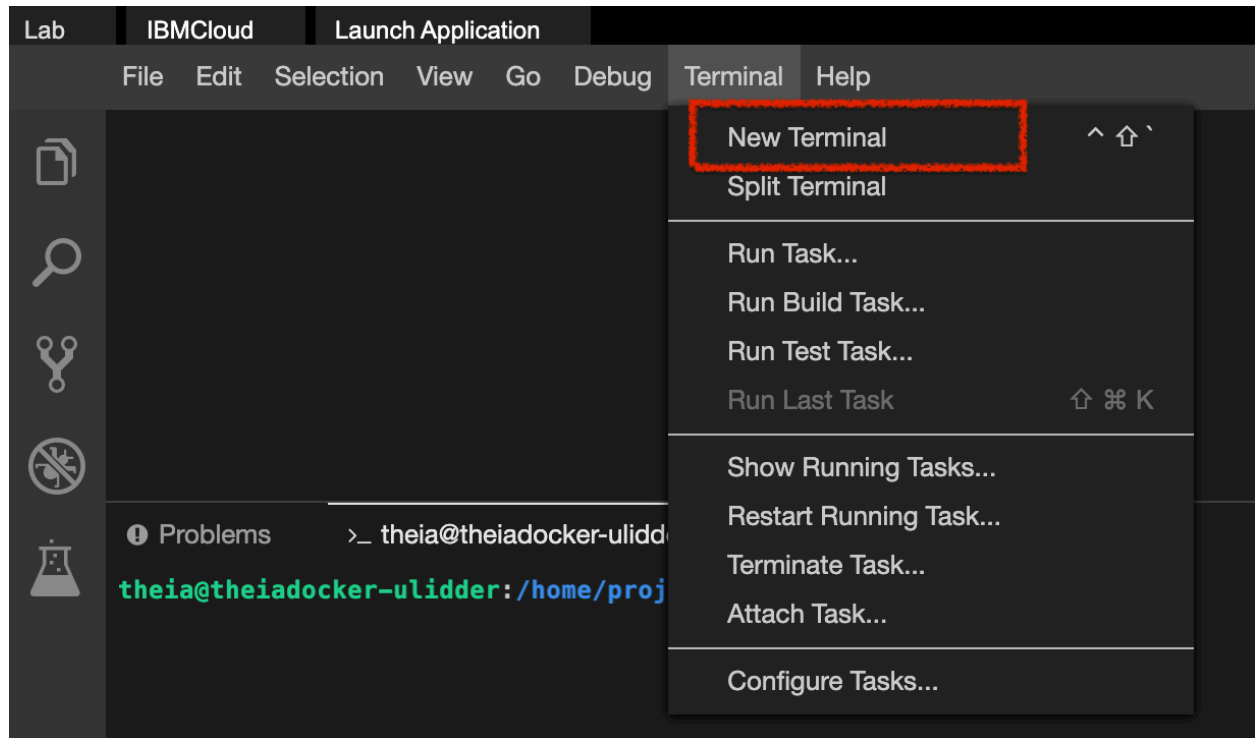
Objectives

After completing this lab you will be able to:

- Install pylint package
- Run Static Code Analysis on a python program
- Check the compliance score of a python program.
- Fix common mistakes and improve the compliance score.

Install the pylint package

1. Open a new terminal.



2. At the terminal run the following command to install pylint.

```
<> plaintext 
```

```
pip3 install pylint==2.11.1
```

3. This should install the pylint package.

```

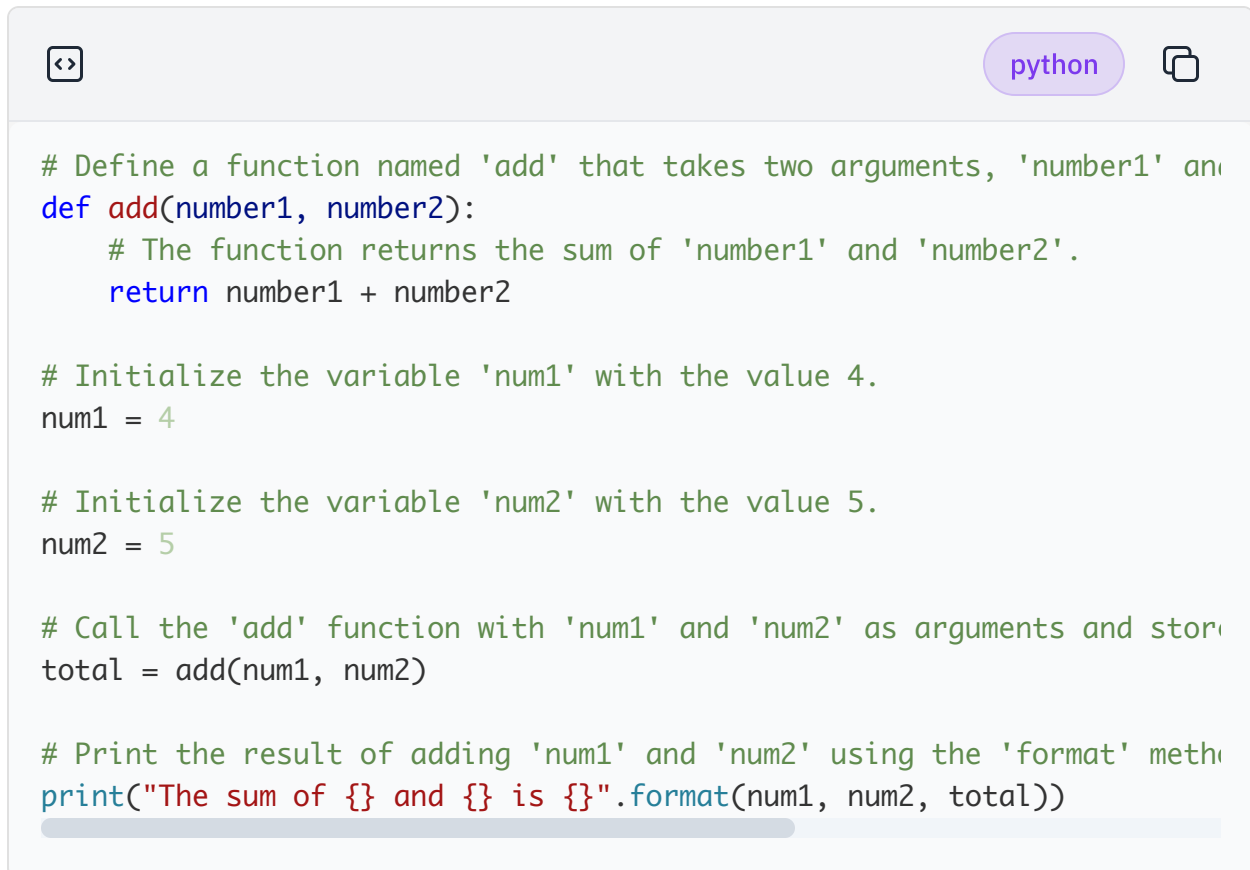
theia@thei :/home/project$ pip3 install pylint
/usr/lib/python3/dist-packages/secretstorage/dhcrypto.py:15: CryptographyDeprecationWarning: int_from_bytes is deprecated, use int.from_bytes instead
  from cryptography.utils import int_from_bytes
/usr/lib/python3/dist-packages/secretstorage/util.py:19: CryptographyDeprecationWarning: int_from_bytes is deprecated, use int.from_bytes instead
  from cryptography.utils import int_from_bytes
Defaulting to user installation because normal site-packages is not writeable
Collecting pylint
  Downloading pylint-2.9.3-py3-none-any.whl (372 kB)
    | 372 kB 17.4 MB/s
Requirement already satisfied: mccabe<0.7,>=0.6 in /usr/local/lib/python3.6/dist-packages (from pylint) (0.6.1)
Collecting isort<6,>=4.2.5
  Downloading isort-5.9.2-py3-none-any.whl (105 kB)
    | 105 kB 43.2 MB/s
Collecting astroid<2.7,>=2.6.2
  Downloading astroid-2.6.2-py3-none-any.whl (228 kB)
    | 228 kB 39.7 MB/s
Requirement already satisfied: toml<=0.7.1 in /usr/local/lib/python3.6/dist-packages (from pylint) (0.10.2)
Collecting wrapt<1.13,>=1.11
  Downloading wrapt-1.12.1.tar.gz (27 kB)
Requirement already satisfied: typing-extensions>=3.7.4 in /home/theia/.local/lib/python3.6/site-packages (from astroid<2.7,>=2.6.2->pylint) (3.7.4.3)
Collecting typed-ast<1.5,>=1.4.0
  Downloading typed_ast-1.4.3-cp36-cp36m-manylinux1_x86_64.whl (743 kB)
    | 743 kB 36.6 MB/s
Requirement already satisfied: lazy-object-proxy>=1.4.0 in /home/theia/.local/lib/python3.6/site-packages (from astroid<2.7,>=2.6.2->pylint) (1.4.3)
Building wheels for collected packages: wrapt
  Building wheel for wrapt (setup.py) ... done
  Created wheel for wrapt: filename=wrapt-1.12.1-cp36-cp36m-linux_x86_64.whl size=69407 sha256=200a0571ea2dccc2792d5d50c31cf2edb755b3bcd14154e5b6a0171266f60f85
  Stored in directory: /home/theia/.cache/pip/wheels/32/42/7f/23cae9ff6ef66798d00dc5d659088e57dbba01566f6c60db63
Successfully built wrapt
Installing collected packages: wrapt, typed-ast, isort, astroid, pylint
Successfully installed astroid-2.6.2 isort-5.9.2 pylint-2.9.3 typed-ast-1.4.3 wrapt-1.12.1

```

Create a sample python file for static code analysis

Create a new file named **sample1.py**

Copy and paste the below code into **sample1.py**

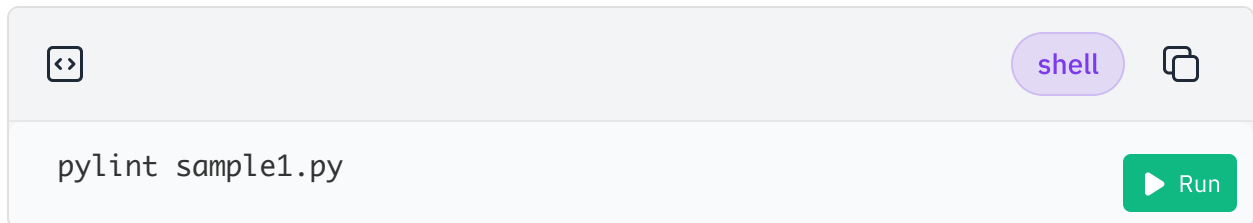
A code editor window with a light gray header bar. On the left is a code icon (two arrows pointing out of a box). On the right is a purple pill-shaped button with the text "python" and a copy icon (two overlapping squares). The main area contains Python code with syntax highlighting: comments are green, keywords are blue, and strings are red. The code defines an 'add' function, initializes 'num1' and 'num2', calls the function, and prints the result.

```
# Define a function named 'add' that takes two arguments, 'number1' and 'number2' and returns the sum of 'number1' and 'number2'.  
def add(number1, number2):  
    # The function returns the sum of 'number1' and 'number2'.  
    return number1 + number2  
  
# Initialize the variable 'num1' with the value 4.  
num1 = 4  
  
# Initialize the variable 'num2' with the value 5.  
num2 = 5  
  
# Call the 'add' function with 'num1' and 'num2' as arguments and store the result in 'total'.  
total = add(num1, num2)  
  
# Print the result of adding 'num1' and 'num2' using the 'format' method.  
print("The sum of {} and {} is {}".format(num1, num2, total))
```

Save the file **sample1.py**

Run pylint

- Open a terminal
- Run the below command

A mockup of a terminal window. The top bar is light gray and contains a code icon on the left, a purple pill-shaped button with the text 'shell' in the center, and a copy icon on the right. The main area of the terminal is white and contains the text 'pylint sample1.py'. In the bottom right corner of the main area, there is a green button with a white play icon and the text 'Run'.

```
pylint sample1.py
```

- Pylint goes through every line of code and gives you a list all the non-compliant lines.
- Pylint gives you a compliance score (10 being maximum).

Correct the mistakes identified by pylint.

- Based on the report given by pylint, changes were made to this code to address the following issues.
 - Missing module-level docstring (`missing-module-docstring`)
 - Missing function docstring (`missing-function-docstring`)
 - Invalid constant naming — module-level variables should follow `UPPER_CASE` naming convention (`invalid-name`)
 - Line too long — one of the comment lines exceeds the default 100-character limit (`line-too-long`)
 - Missing final newline at the end of the file (`missing-final-newline`)
 - Use of `.format()` where an f-string is preferred (`consider-using-f-string`)

***Note on constant naming:** Pylint treats all variables defined at the module level (outside of functions or classes) as constants and expects them to follow `UPPER_CASE` naming. This applies even to variables that store the result of a function call, such as `total`. To satisfy pylint, either name them in `UPPER_CASE` (e.g., `TOTAL`) or add an inline `# pylint: disable=invalid-name` comment on that line.*

- Create a new file named **sample2.py**
- Copy and paste the below code into **sample2.py**



python



```
"""This module demonstrates a simple addition operation using a helper

# Define a function named 'add' that takes two arguments, 'number1' and
# The purpose of this function is to add the two numbers and return the
def add(number1, number2):
    """Return the sum of number1 and number2."""
    # Return the sum of 'number1' and 'number2'.
    # This line computes the addition of the two input numbers and outputs
    return number1 + number2

# Initialize the constant variable 'NUM1' with the value 4.
# Constants are usually written in uppercase letters to indicate that
NUM1 = 4

# Initialize the constant variable 'NUM2' with the value 5.
# This variable will be used as the second input to the 'add' function
NUM2 = 5

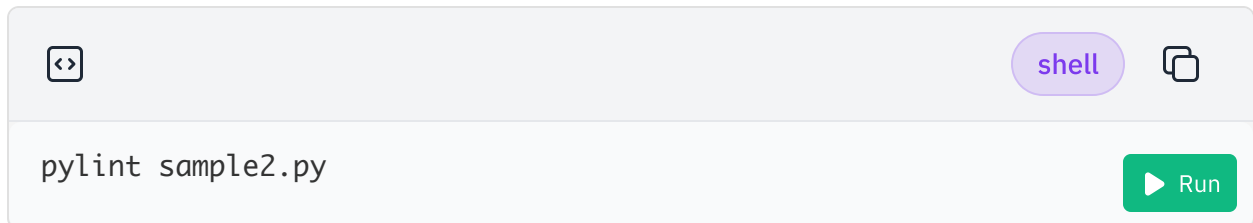
# Call the 'add' function with 'NUM1' and 'NUM2' as arguments.
# The result of this addition operation is stored in the variable 'TOTAL'
TOTAL = add(NUM1, NUM2)

# Print a formatted string that displays the sum of 'NUM1' and 'NUM2'.
# An f-string is used to insert the values of 'NUM1', 'NUM2', and 'TOTAL'
print(f"The sum of {NUM1} and {NUM2} is {TOTAL}")
```

Save the file **sample2.py**

Run pylint

- Open a terminal
- Run the below command

A mockup of a terminal window. The top bar is light gray and contains a code icon on the left, a purple pill-shaped button with the text 'shell' in the center, and a copy icon on the right. The main area of the terminal is white and contains the text 'pylint sample2.py'. In the bottom right corner of the main area, there is a green button with a white play icon and the text 'Run'.

```
pylint sample2.py
```

- This will give you the compliance score.
- This time you should see the score improve significantly — aiming for a perfect 10.

Your task

Improve the score in sample2.py to a perfect 10 by correcting all the issues pointed out by pylint. If you can't figure out how to solve some issues it is helpful to google the pylint message.

► Hints for common issues:

Congratulations!

You now know how to perform static code analysis.

Author(s)

Ramesh Sannareddy

Other Contributors

Rav Ahuja

© IBM Corporation. All rights reserved.

Hands-on Lab: Static Code Analysis



Estimated time needed: **30** minutes

Objectives

After completing this lab you will be able to:

- Install pylint package

- Check the compliance score of a python program.
- Fix common mistakes and improve the compliance score.

Install the pylint package →